



Sri Eshwar
College of Engineering
An Autonomous Institution



Name _____

Roll No. _____

Reg No. _____

Class _____

U23EC383 – Embedded Systems and IoT Laboratory

—————
— **RECORD NOTE BOOK** —



Sri Eshwar
College of Engineering
An Autonomous Institution
Affiliated to Anna University



Kondampatti (Post), Vadasithur (Via), Coimbatore - 641 202.

Department of

.....

Record work of Laboratory

Certified bonafide Record of work done by

Name : Roll No. :

Class : Branch :

Reg. No. :

Place : HOD Faculty In charge

Date :

University Register No.

Submitted for the University Practical Examination held on

.....

Internal Examiner

External Examiner

Instructions for LABORATORY Classes

- ❖ Enter the Lab with observation, record notebook, calculator & necessary tools
- ❖ Enter the Lab with **CLOSED FOOTWEAR**
- ❖ Maintain Silence during the Lab Hours
- ❖ Boys should **"TUCK IN"** the Shirts
- ❖ Girls are instructed to wear **OVERCOATS**
- ❖ **HANGING CHAINS, RINGS, WRIST WATCHES** and like items are likely to cause accidents and hence are to be **AVOIDED**
- ❖ **LONG HAIR** should be protected, let it not be loose especially near **ROTATING MACHINERY**
- ❖ Any machine / equipment **SHOULD NOT BE OPERATED** other than the prescribed one for the day
- ❖ Read and follow the work instructions inside the laboratory
- ❖ **POWER SUPPLY** to your table should be obtained only through the **LAB TECHNICIAN**
- ❖ Do not **LEAN** and do not be **CLOSE** to the rotating components
- ❖ Handle the equipments with care
- ❖ **TOOLS & GAUGE** sets are to be **RETURNED** before leaving the lab
- ❖ **ALL SUB - HEADING** and **DETAILS** should be neatly written on the right hand side page of the record
- ❖ For example
 - Aim of the experiment
 - Apparatus / Tools / Instruments required
 - Procedure
 - Model calculation
 - Results / discussions / inference
- ❖ The sketches, block diagrams, circuits and flow charts are to be drawn on the left side blank page
- ❖ For example
 - Neat Diagram
 - Specifications / Design Details
 - Tabulations
 - Graph
- ❖ **GRAPHS / FIGURES** drawn on separate sheets in special cases, should be firmly pasted on to the record.
- ❖ Before writing the record, the student should get the corresponding observations signed by the **FACULTY - IN- CHARGE**
- ❖ The observation and record should be completed in all respects and submitted in the very **NEXT CLASS** itself.
- ❖ Begin every experiment on a **NEW PAGE**
- ❖ Write the **NAME OF EXPERIMENT** in Capital letters on the top of the page
- ❖ Experiment number with date should be written at the top left hand corner
- ❖ Be **PATIENT, STEADY, SYSTEMATIC** and **REGULAR**
- ❖ Follow the dress code for Laboratory classes
- ❖ Maintain punctuality

INDEX

S.No	Date	Name of the Experiment	Page No.	Total (75)	Faculty sign

Signature of the Faculty Member

..... Laboratory

Name of the Student :

Roll No :

Year :

Semester :

<i>Description</i>	<i>I Month</i>	<i>II Month</i>	<i>III Month</i>
<i>Number of Experiments Proposed</i>			
<i>Number of Experiments done</i>			
<i>Reason for Deviation (if any)</i>			
<i>Remark by the Faculty Member</i>			

Vision of the Department

To build a conducive academic and research environment in the stream of Artificial Intelligence and Data Science for enabling global education, research and entrepreneurship.

Mission of the Department

- Update and enrich the curriculum by integrating global trends and technological advancements to ensure the academic excellence.
- Promote a research-driven environment by providing access to state-of-the-art infrastructure with emerging technologies.
- Empower the faculty through continuous upskilling, international collaborations, and active research engagement to provide globally competent education.
- Encourage student-centric, industry-aligned learning that nurtures innovation and entrepreneurship activities for solving real-world and societal challenges.

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop the solution of **complex engineering problems**.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze **complex engineering problems** reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for **complex engineering problems** and design/develop systems/components/processes to meet identified needs with consideration for public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of **complex engineering problems** using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve **complex engineering problems**. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving **complex engineering problems** for its impact on sustainability with reference to the economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Teamwork: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Educational Objectives (PEOs)

PEO1	Build a progressive career in industry or pursue higher education or succeed as an entrepreneur.
PEO2	Excel in cutting-edge technologies of Artificial Intelligence and Data Science to address the real-world challenges.
PEO3	Exhibit professional ethics, involvement in team work in their profession and contributing to the advancement of society.

Program Specific Outcomes (PSOs)

PSO1	Apply programming, problem-solving, and hardware integration skills with AI and data science to design intelligent, secure and efficient systems.
PSO2	Develop an AI-driven application by exploring optimized coding competencies using predictive modeling and data analytics technologies

Course Outcomes (Cos):

U23EC383.1	Analyze and synthesize the architecture of Embedded Systems, delineating its constituent components.
U23EC383.2	Analyze the design process of embedded systems by understanding the core concepts and evaluating system requirements to implement efficient and optimized designs.
U23EC383.3	Apply advanced cognitive skills to conceptualize the architecture of IoT systems and develop basic applications employing embedded hardware.
U23EC383.4	Apply higher-order cognitive abilities to implement Cloud services in IoT applications
U23EC383.5	Apply NodeMCU and Raspberry Pi boards to design and develop IoT applications by integrating hardware and software components effectively.

CO & PO/PSO Mapping:

COURSE	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
U23EC3 83.1	3	3	3	1	-	-	-	-	1	-	1	-	-
U23EC3 83.2	3	3	3	1	-	-	-	-	1	-	1	-	-
U23EC3 83.3	3	3	3	1	3	-	-	-	1	-	2	2	-
U23EC3 83.4	3	3	3	1	3	-	-	-	1	-	2	2	-
U23EC3 83.5	3	3	3	1	3	-	-	-	1	-	2	2	-

List of Experiments

S.No.	Name of the Experiment
1.	Interfacing Arduino with Pot and Servo motor.
2.	Interfacing Arduino with Stepper Motor.
3.	Interfacing Arduino with Analog/Digital sensors.
4.	Write a program to interface LED/Buzzer with Arduino and to turn ON LED for 1 second after every 2 seconds.
5.	Write a program to interface Digital sensor PIR with Arduino and to turn ON LED when motion is detected.
6.	Write a program to interface DHT22 sensor with Arduino and display temperature and humidity readings.
7.	Write a program to interface the motor using relay with Raspberry Pi and turn ON the motor when the temperature is high.
8.	Write a program to interface LCD with Raspberry Pi and print temperature and humidity readings on it.
9.	Write a program to interface a flame/smoke sensor with Arduino/Raspberry Pi and give an alert message when flame/smoke is detected.
10.	Implement any case study using Arduino/Raspberry Pi.

Safety Precautions:

- Do not touch any exposed wires. Work with a dry hand.
- Do not open up equipment casing.
- Do not unplug the cable while the power is switched on.
- Do not wear long, loose ties, scarves or other loose clothing. Make sure that long hairs are tied.

EXP NO : 1	Interfacing Arduino with Pot and Servo motor
DATE :	

Aim:

To control the position of a servo motor based on the analog input from a potentiometer by mapping the sensor values to servo rotation angles.

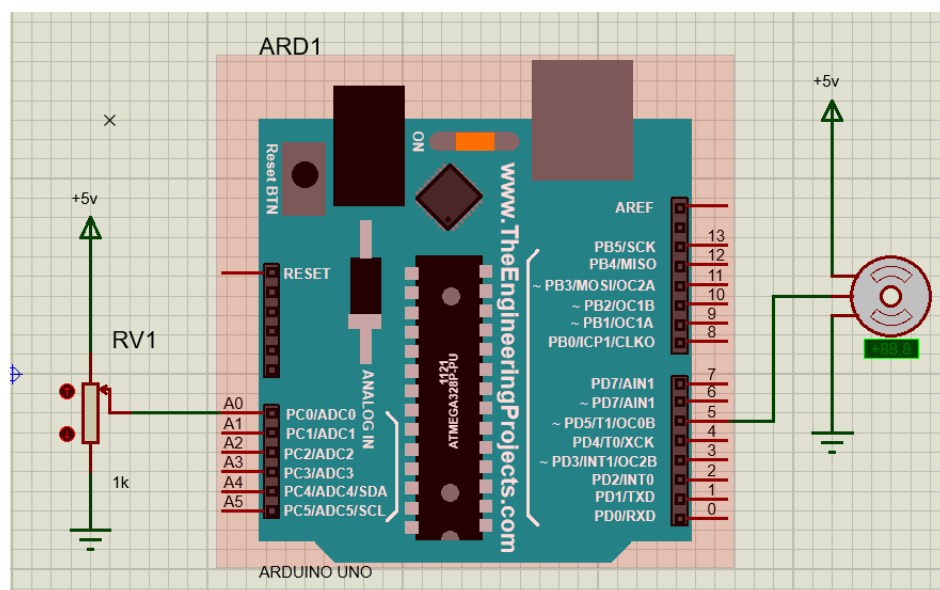
Components & Software Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. Arduino Uno Board
4. Potentiometer
5. Servo Motor

Theory:

A servo motor rotates to a specific angle between 0° and 180° based on the PWM signal sent from the Arduino. The potentiometer provides an analog voltage input between 0V and 5V, read by the Arduino as a value between 0 and 1023. This analog value is mapped linearly to the servo's rotation range (0 to 180 degrees). The servo motor moves to the corresponding angle based on the potentiometer position.

Schematic Diagram:



Procedure:

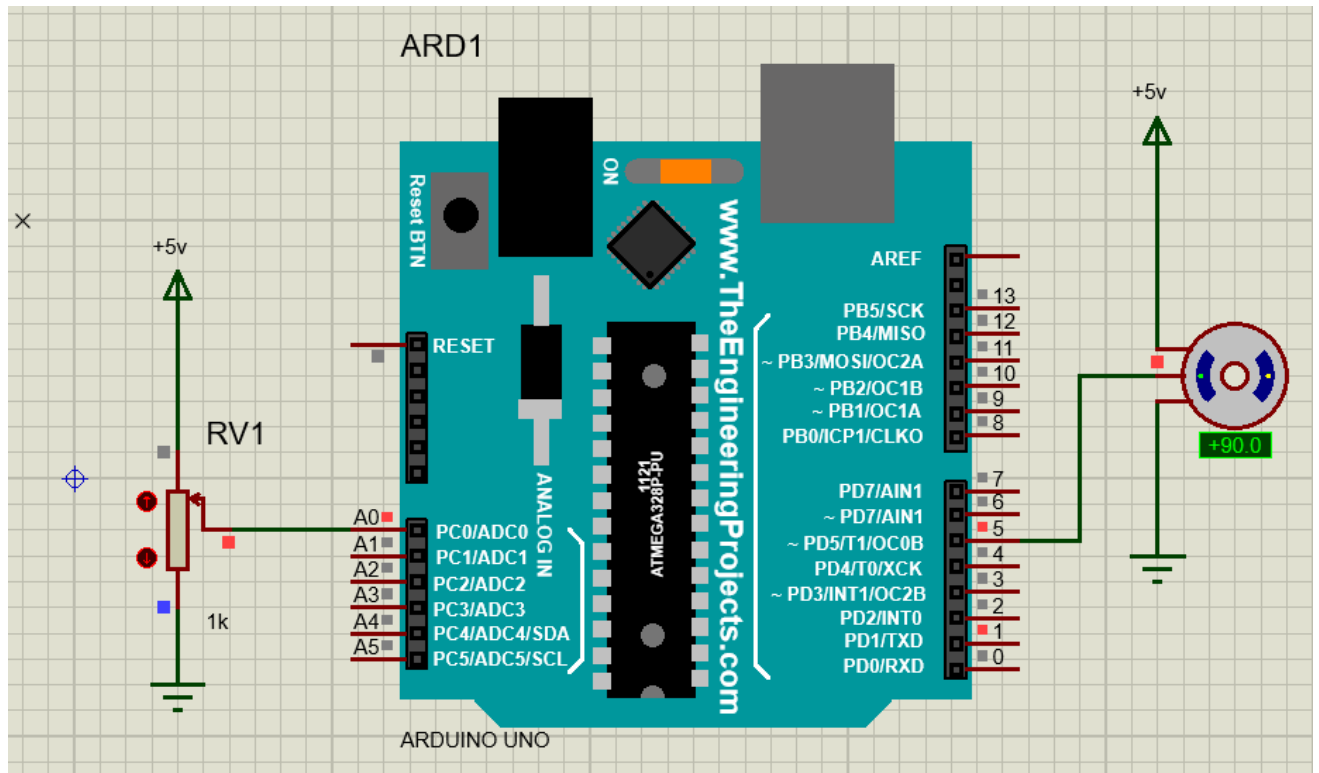
1. Connect the servo motor signal wire to digital pin 2 on the Arduino.
2. Connect the potentiometer middle pin to analog pin A0 on the Arduino, and the other two pins to 5V and GND.
3. Connect the servo motor's power and ground pins to 5V and GND respectively.
4. Upload the program to the Arduino.
5. Rotate the potentiometer knob; the Arduino reads the analog value.
6. The analog value is mapped to a rotation angle between 0° and 180°.
7. The servo motor shaft rotates to the mapped angle, mimicking the potentiometer position.
8. The servo rotates proportionally as the potentiometer knob is turned.

Code:

```
#include <Servo.h>
Servo myservo;
void setup()
{
    myservo.attach(5);
    Serial.begin(9600);
    pinMode(A0,INPUT);
}

void loop()
{
    int data=analogRead(A0);
    int mydata=map(data,0,1023,0,180); //map(value, from Low, from High, to Low, to High)
    myservo.write(mydata);
    // mapping of 2 devices pot and servo motor
    // pot from 0-1023(1024), servo motor vary from 0-180 degree
}
```

Output:



Result:

EXP NO : 2	Interfacing Arduino with Stepper Motor
DATE :	

Aim:

To control the speed of a stepper motor by varying the motor speed dynamically using the Arduino Stepper library.

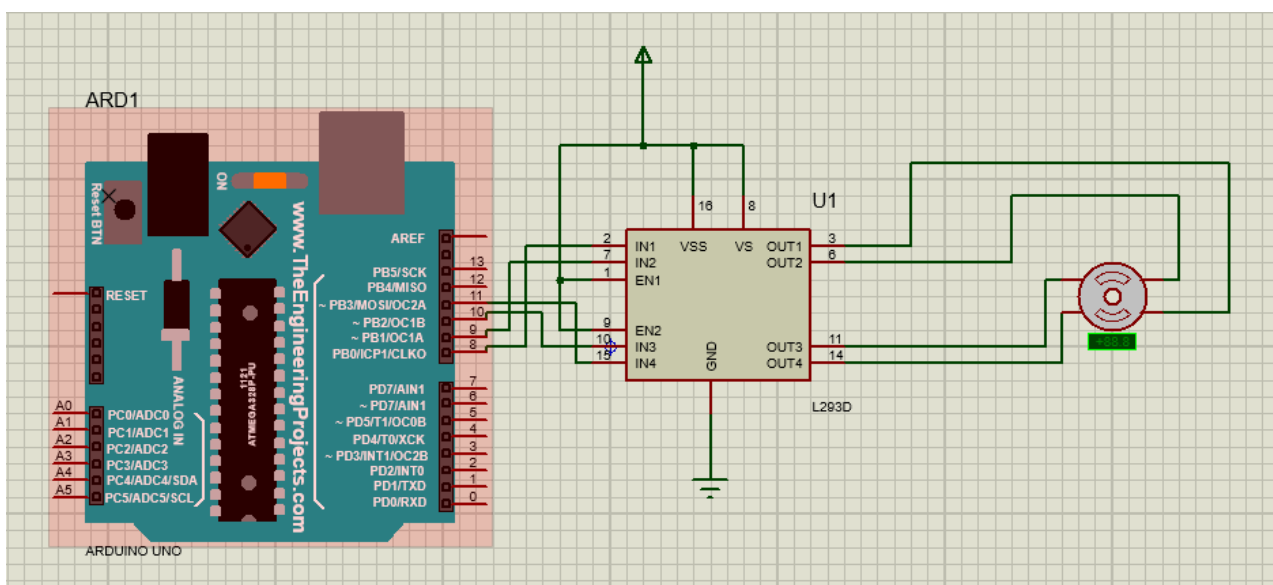
Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. Arduino Uno Board
4. Stepper Motor Driver L239D
5. Stepper Motor

Theory:

A Stepper Motor moves in discrete steps, controlled by digital pulses. The Stepper library in Arduino helps manage the sequence of pulses required to rotate the motor in precise steps. The analog sensor connected to pin A0 provides a varying voltage which is read as an analog value between 0 and 1023. The sensor reading is mapped to a speed value (in RPM) to dynamically control the speed of the stepper motor. The motor speed is updated continuously based on sensor input, allowing smooth speed control.

Schematic Diagram:



Procedure:

1. Connect the stepper motor to Arduino pins 8, 9, 10, and 11 through the driver module.
2. Connect the analog sensor (e.g., potentiometer) to analog input pin A0.
3. Power the stepper motor with an appropriate external power supply if needed.
4. Upload the program to the Arduino board.
5. Rotate or adjust the sensor connected to A0 to change the analog reading or use code to control the motor instead of analog input(potentiometer).
6. The stepper motor speed will change accordingly; the higher the sensor value, the faster the motor spins.
7. Observe the motor rotating at speeds proportional to the sensor input.

Code:

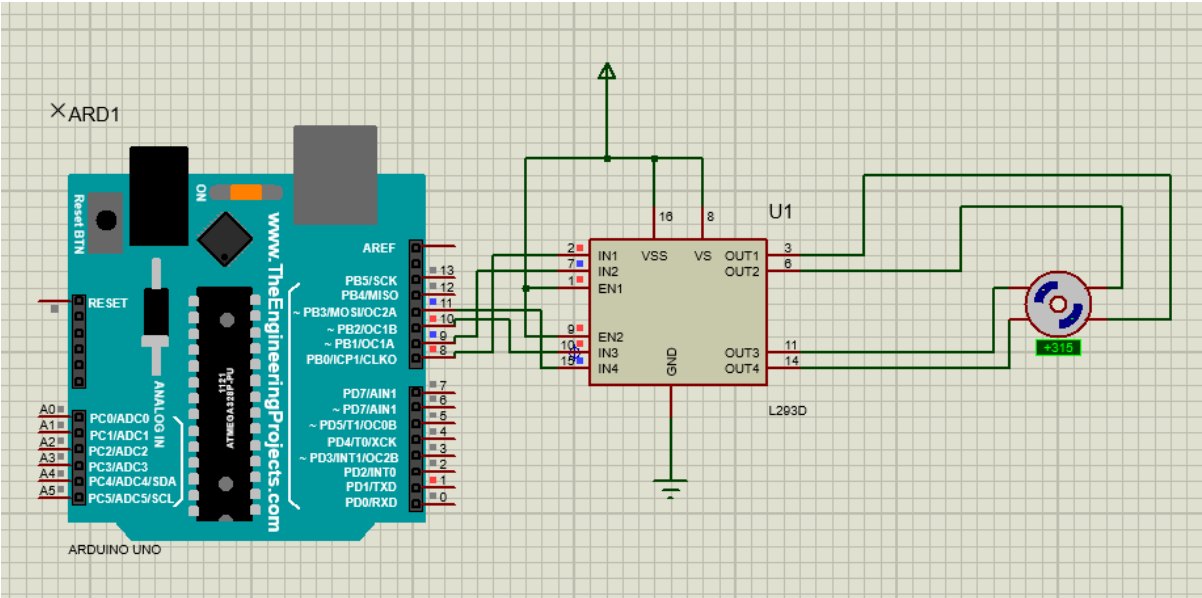
```
//stepper motor interfacing with Arduino in proteus
#include <Stepper.h>

const int apr =40;
Stepper steper(apr, 8, 9, 10, 11);

void setup()
{
    steper.setSpeed(10);
    //initialize the serial port
    Serial.begin(9600);
}

void loop()
{
    steper.step(apr);
    delay(1000);
    steper.step(-apr);
    delay(1000);
}
```

Output:



Result:

EXP NO : 3	Interfacing Arduino with Analog and Digital sensors
DATE :	

Aim:

To control a control LED based on the LDR (Light Dependent Resistor -Analog sensor) sensor such that the LED is activated only when ambient light level is below a certain threshold.

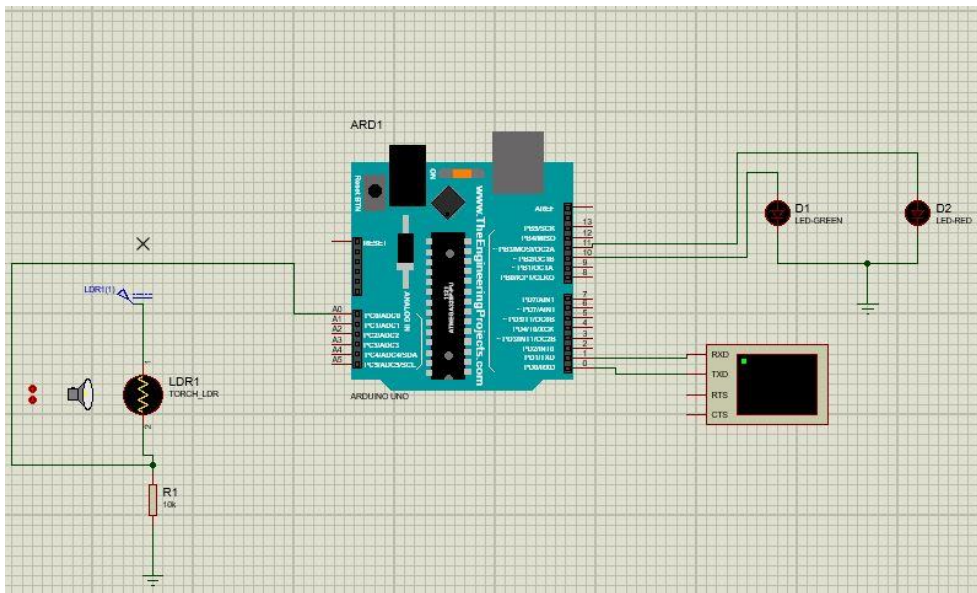
Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. LDR
4. LEDs (Green, Red)
5. Serial Monitor

Theory:

A Light Dependent Resistor (LDR) is a light-sensitive device whose resistance varies inversely with the intensity of incident light, making it suitable for sensing ambient light conditions. In this setup, the LDR is interfaced with a microcontroller or comparator circuit to convert light intensity into an electrical signal. A predefined threshold value is used to differentiate between low-light and high-light environments. When the measured light level falls below the threshold, it indicates a dark condition, and the Green LED is activated. When the light level rises above the threshold, indicating a bright condition, the Red LED is turned ON. This system illustrates automatic control and decision-making based on real-time sensor input.

Schematic Diagram:



Procedure:

1. Connect the LEDs to the output pin of 10 and 11 of Arduino.
2. Connect the LDR sensor as a voltage divider circuit to analog pin A0 of Arduino (LDR connected in series with a resistor).
3. Upload the provided code to the Arduino board.
4. Open the Serial Monitor to observe LDR sensor readings.
5. When the environment is dark (LDR value below threshold 500), the output Green LED glows and when the environment is bright (LDR value above threshold 500), the output Red LED will glow.
6. If the light level is high or low, the corresponding LED will glow accordingly.

Code:

```
// interface LDR -analog sensor with Arduino
int x;
void setup()
{
  pinMode(10,OUTPUT); // Pins Connected To Led Where Set In As Output Mode:
  pinMode(11, OUTPUT);
  Serial.begin(9600);
}
```

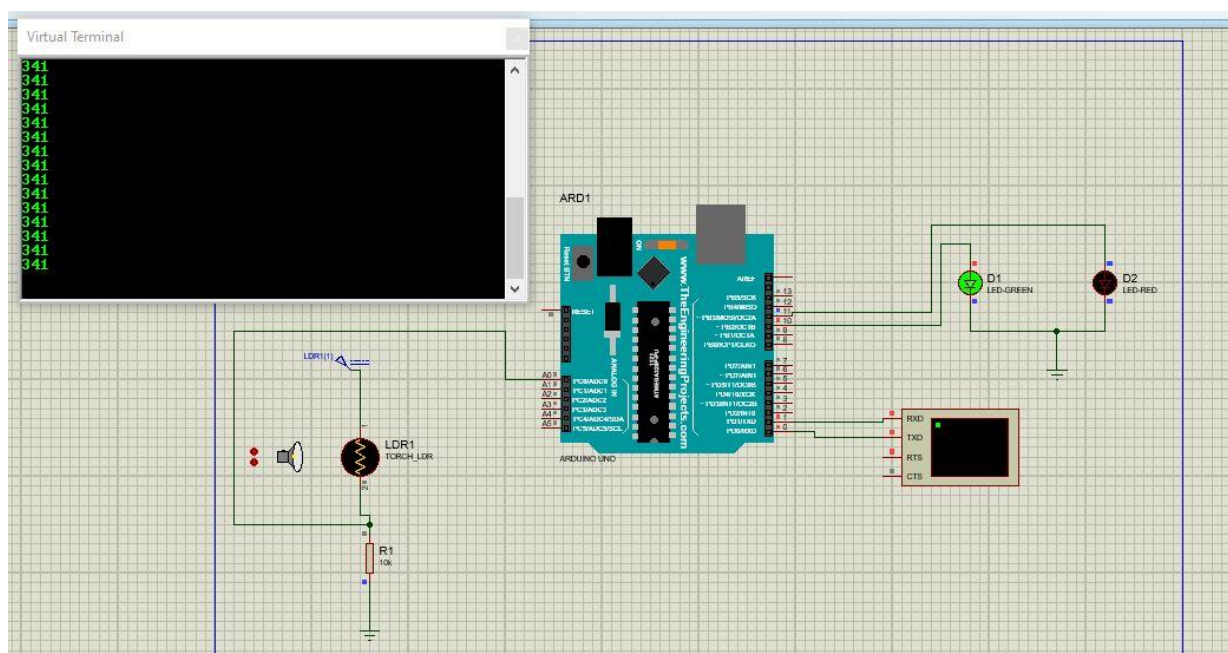
```

void loop()
{
    x= analogRead(A0); // enter analog value between 0 and 1023 representing voltage from analog pin A0:
    Serial.println(x);
    if (x<500)
    {
        digitalWrite(10,HIGH); //if value converted between <500 green LED will be ON & RED LED will be OFF
        digitalWrite(11,LOW);
    }
    if (x>500)
    {
        digitalWrite(10,LOW); //if value converted between >500 green LED will be OFF & RED LED will be ON
        digitalWrite(11,HIGH);
    }
    delay(200); // loop is executed at high speed
}

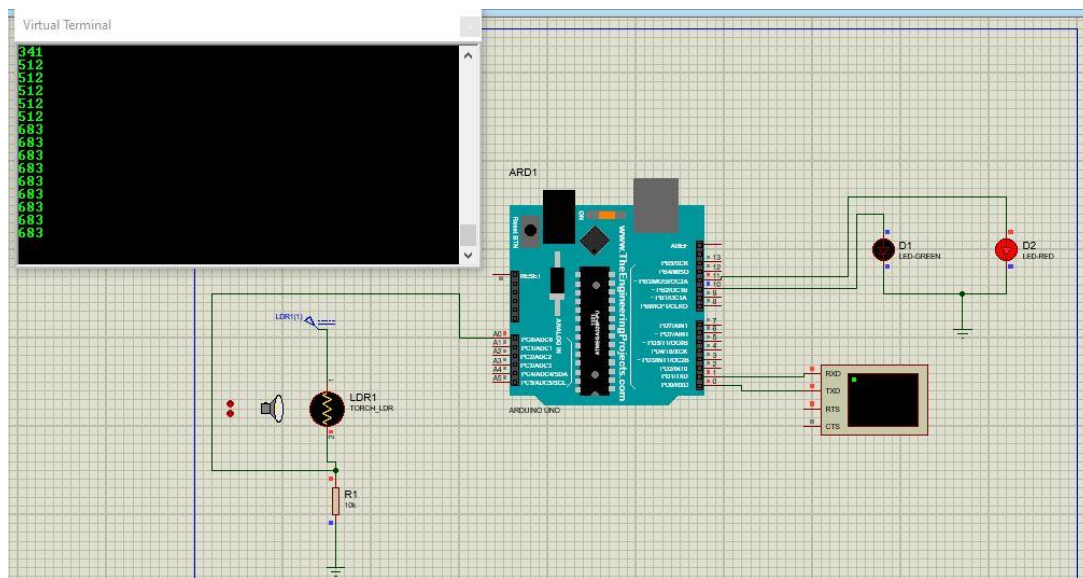
```

Output:

When the Threshold value is < 500 Green LED will be activated



When the Threshold value is > 500 Green LED will be activated



Result:

Aim:

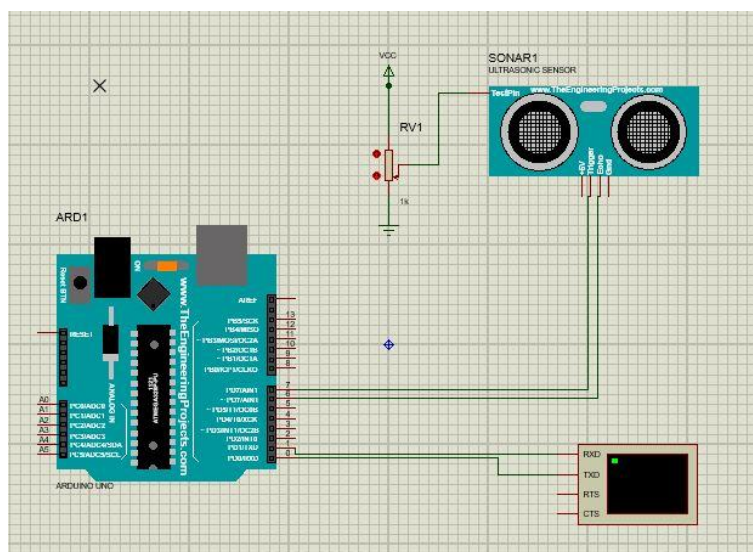
To measure the distance from the target based on the Ultrasonic waves (Ultrasonic sensor-Digital Sensor).

Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. Ultrasonic Sensor (Digital Sensor)
4. Potentiometer
5. Serial Monitor

Theory:

An Ultrasonic Sensor is a device used to measure distance by emitting ultrasonic sound waves and detecting their reflection from nearby objects. It operates on the principle of time-of-flight, where the time taken for the sound wave to travel to an object and return is used to calculate the distance. The sensor consists of a transmitter that sends ultrasonic pulses and a receiver that captures the reflected echoes. This information is processed by a microcontroller to determine the distance of objects in front of the sensor. Ultrasonic sensors are widely used in obstacle detection, robotics, and automation systems. They provide a non-contact method of measurement, making them reliable in various environmental conditions.

Schematic Diagram:

Procedure:

1. Connect the ultrasonic sensor trigger and echo pin to 7 and 6 of Arduino.
2. Connect the Potentiometer to Test pin of Ultrasonic sensor (To vary the distance level).
3. Upload the provided code to the Arduino board.
4. Upload the corresponding library file to ultrasonic sensor.
5. Open the Serial Monitor to observe ultrasonic sensor readings.
6. Based on the variation in the potentiometer, the corresponding distance will be displayed in the serial monitor.

Code:**// Interface Ultrasonic Sensor with Arduino**

```
const int pingPin=7; // Trigger pin of Ultrasonic sensor
const int echoPin=6; // Echo pin of Ultrasonic Sensor
void setup()
{
  Serial.begin(9600); //Starting Serial Terminal
}
void loop()
{
  long duration, inches, cm;
  pinMode(pingPin,OUTPUT);
  digitalWrite(pingPin, LOW);
  delayMicroseconds(2);
  digitalWrite(pingPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(pingPin, LOW);

  pinMode(echoPin, INPUT);
  duration =pulseIn(echoPin,HIGH);
  inches=microsecondsToInches(duration);
  cm=microsecondsToCentimeters(duration);
```

```

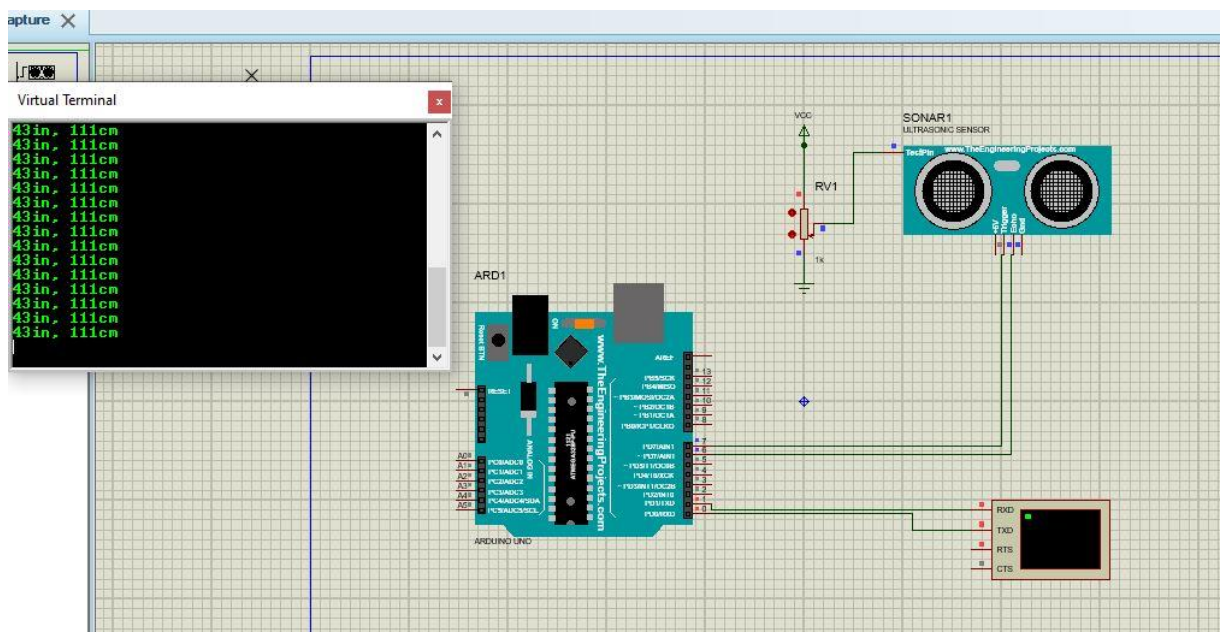
Serial.print(inches);
Serial.print("in, ");
Serial.print(cm);
Serial.print("cm");
Serial.println();
delay(100);
}

long microsecondsToInches(long microseconds)
{
    return microseconds / 74 / 2;
}

long microsecondsToCentimeters(long microseconds)
{
    return microseconds / 29 / 2;
}

```

Output:



Result:

EXP NO : 4	Interfacing LED/Buzzer with Arduino
DATE :	

Aim:

To interface an LED/Buzzer with Arduino and to turn ON the LED for 1 second after every 2 seconds.

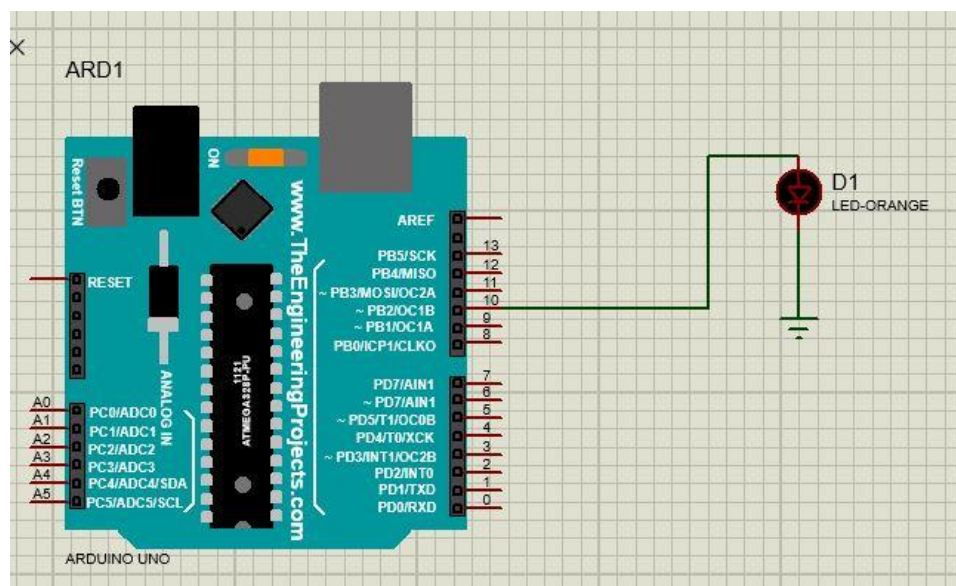
Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. LED

Theory:

An LED (Light Emitting Diode) is an electronic component that emits light when current passes through it. A Buzzer converts electrical energy into sound through vibration of a diaphragm. The Arduino UNO controls the LED or buzzer using its digital output pins. By writing a simple program, we can switch the output pin HIGH (to turn ON LED) and LOW (to turn OFF LED) with a time delay. The delay() function in Arduino introduces a pause in milliseconds (1000 ms = 1 second).

Schematic Diagram:



Procedure:

1. Connect the LED to the pin 10 of Arduino UNO Board.
2. Upload the provided code to the Arduino board.
3. Observe that LED glows for 1 sec for every for 2 second

Code:

//LED/Buzzer with Arduino and to turn ON LED for 1 second after every 2 seconds.

```
const int LED=10;
```

```
void setup()
```

```
{
```

```
  pinMode(LED,OUTPUT);// Connect LED to pin 10 as output:
```

```
}
```

```
void loop()
```

```
{
```

```
  digitalWrite(LED,HIGH);// give LED to HIGH value:
```

```
  delay(1000); //wait for 1000 msec or 1 sec
```

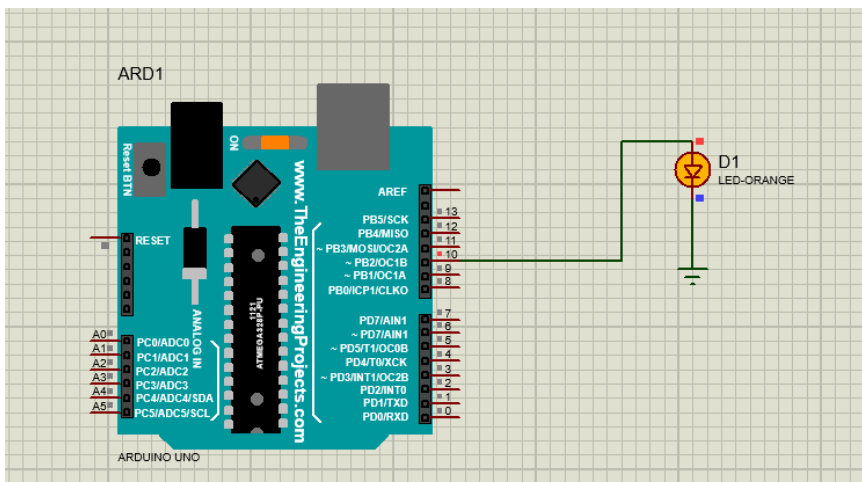
```
  digitalWrite(LED,LOW);// give LED to LOW value:
```

```
  delay(2000); //wait for 2000 msec or 2 sec
```

```
}
```

Output :

LED Turn ON for 1 second after every 2 seconds



Result:

EXP NO : 5	Interfacing Digital sensor - PIR with Arduino
DATE :	

Aim:

To interface a Digital PIR (Passive Infrared) Sensor with Arduino and to turn ON an LED when motion is detected.

Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. PIR (Passive InfraRed- Digital Sensor)
4. Potentiometer
5. Logic State device
6. Serial Monitor

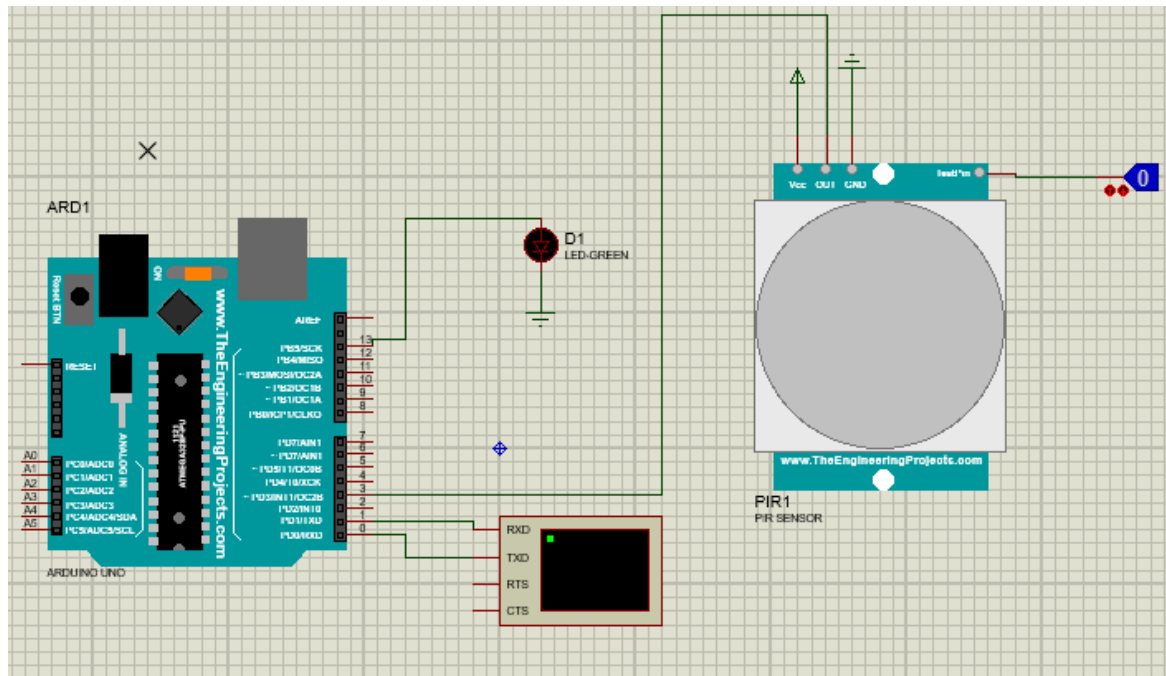
Theory:

The PIR sensor detects infrared radiation emitted by humans or animals. A PIR (Passive Infrared) sensor detects motion by measuring changes in infrared (heat) radiation in its surroundings. i) When a human or animal moves in front of it, the sensor outputs a HIGH (logic 1) signal. ii) When there is no motion, the output remains LOW (logic 0). This principle is widely used in security systems, automatic lighting, and intrusion detection.

The Arduino continuously monitors this digital output signal:

- If **HIGH**, Arduino turns **ON the LED**.
- If **LOW**, Arduino turns **OFF the LED**.

Schematic Diagram:



Procedure:

1. Connect the LED to the pin 13 of Arduino UNO Board.
2. Connect the power and ground pin of PIR sensor to power and ground terminals
3. Connect the logic state device to TEST pin of PIR sensor
4. Upload the provided code to the Arduino board.
5. Upload the corresponding library file with PIR sensor.
6. Open the Serial Monitor to observe PIR sensor data
7. When the logic state is changed to “0”, it indicates that there is no motion detected.
8. When the logic state is changed to “1”, it indicates that there is motion detected.
9. Whenever Motion detected, LED will glow.

Code:

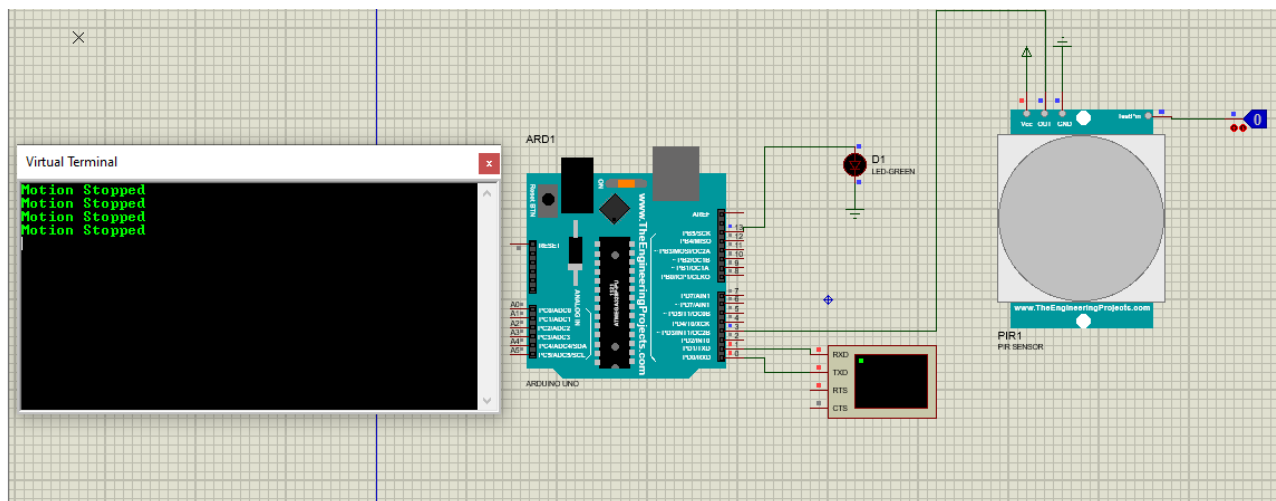
```
//PIR sensor interface with Arduino and LED

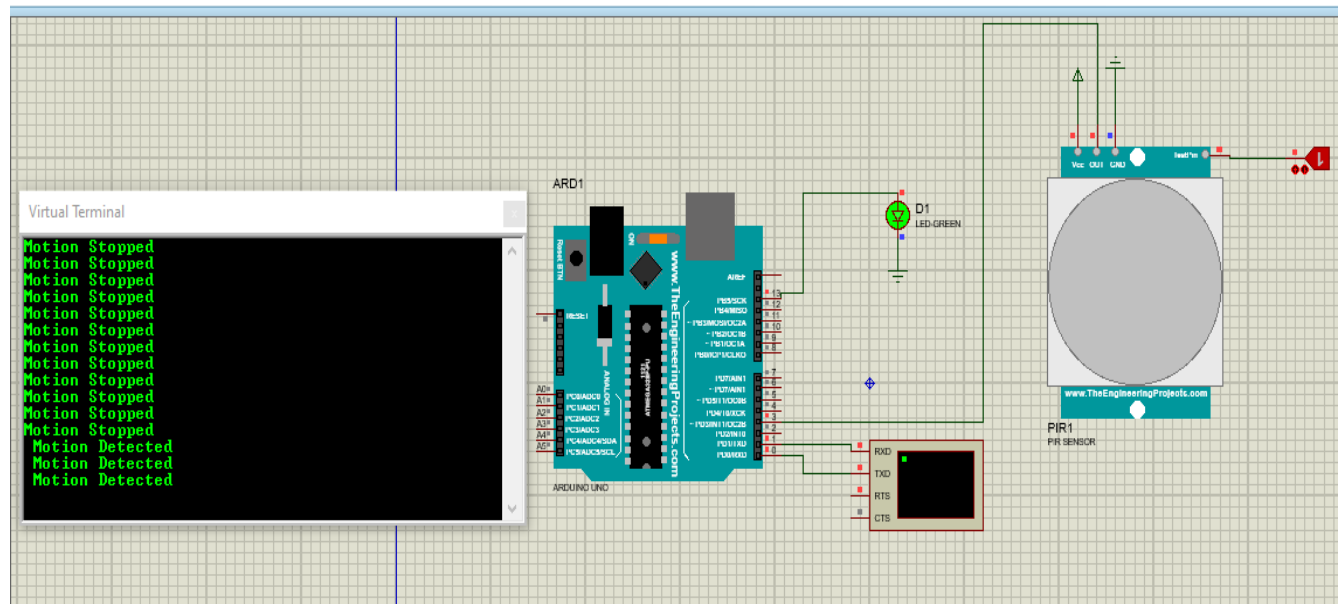
const int pirpin=3; //PIR sensor input pin
const int ledpin=13; // LED output pin

void setup()
{
  pinMode(pirpin,INPUT);// input pin
  pinMode(ledpin,OUTPUT); // LED output pin
  Serial.begin(9600);
}

void loop()
{
  int motionState=digitalRead(pirpin);//Read PIR sensor:
  if(motionState==HIGH){
    digitalWrite(ledpin, HIGH); // Turn ON LED
    Serial.println(" Motion Detected");
  } else
  {
    digitalWrite(ledpin,LOW); //Turn OFF LED
    Serial.println("Motion Stopped");
  }
  delay(1000); // short elay for stability
}
```

Output:





Result:

EXP NO : 6	Interfacing DHT22 Sensor with Arduino
DATE :	

Aim:

To interface a DHT11/DHT22 temperature and humidity sensor with Arduino UNO and display the measured temperature and humidity readings on the serial monitor.

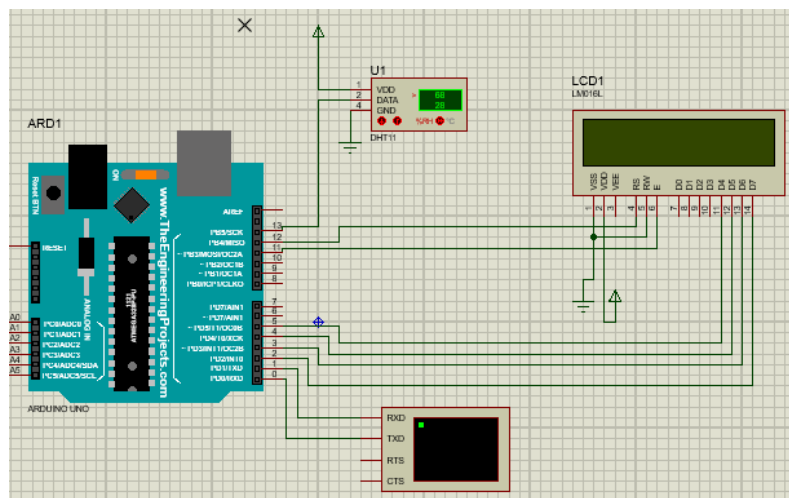
Components Required:

1. Arduino IDE 2.32 or Higher
2. Proteus Professional 8.1 or Higher
3. DHT11/22 Digital sensor
4. LCD (16x2)
5. Serial Monitor

Theory:

The DHT22 (also known as AM2302) is a digital sensor that measures both temperature and relative humidity. It uses a capacitive humidity sensor and a thermistor to measure environmental conditions, then transmits the data through a single-wire digital interface to the Arduino. The Arduino reads this digital signal using the DHT library, which converts raw data into temperature (°C) and humidity (%) values. These readings can be displayed on the Serial Monitor or an LCD display. The Arduino continuously monitors this digital output signal:

Schematic Diagram:



Procedure:

1. Connect the DATA pin of DHT sensor to pin 13 of Arduino UNO Board and VDD and GND to supply and ground respectively.
2. Connect the VSS , RW of LCD(16x2) to ground VCC to power supply.
3. Connect the RS and EN pin of LCD to pin 12 and 11 of Arduino board.
4. Connect the DATA pins D4,D5,D6,D7(LCD) to 5,4,3,2 of Arduino Board.
5. Connect the serial monitor TXD & RXD to Pin 0 and 1 of Arduino board.
6. Upload the provided code to the Arduino board.
7. Open the Serial Monitor to observe temperature and humidity of sensor data.
8. The variation in the DHT11 sensor with temperature and Humidity will be displayed in the LCD and serial monitor.

Code:

//Need to add DHT Library & ADAAfruit_sensor library

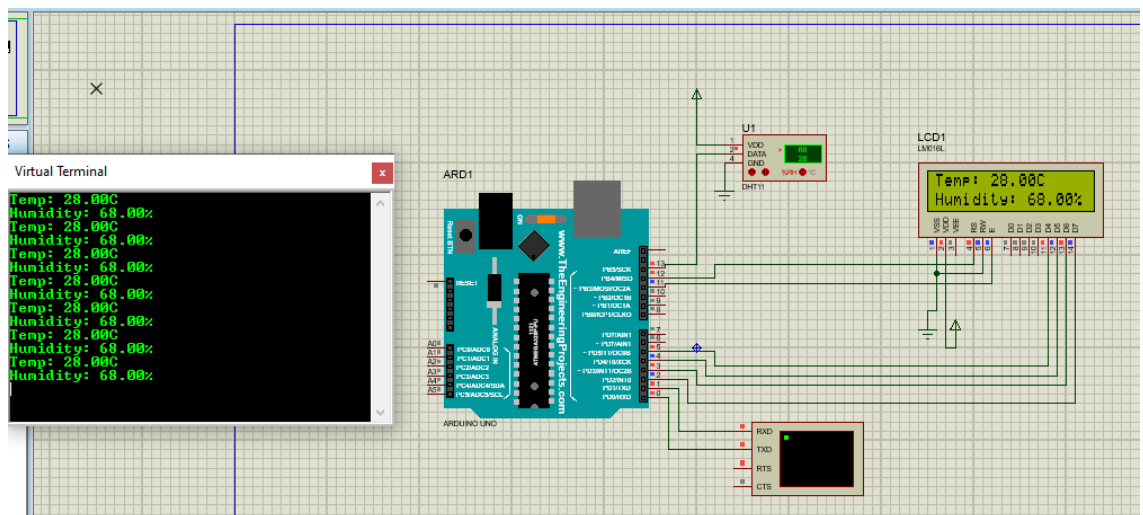
```
#include<DHT.h>
#include<LiquidCrystal.h>
LiquidCrystal lcd(12,11,5,4,3,2);
#define DHT_11PIN 13
DHT dht(DHT_11PIN,DHT11);
void setup()
{
  lcd.begin(16,2);
  dht.begin();
  Serial.begin(9600);
}
void loop()
{
  float temp=dht.readTemperature();
  float hum=dht.readHumidity();
  lcd.setCursor(0,0);
  lcd.print("Temp: ");
  lcd.print(temp);
```

```

lcd.print("C");
lcd.setCursor(0,1);
lcd.print("Humidity: ");
lcd.print(hum);
lcd.print("%");
Serial.print("Temp: ");
Serial.print(temp);
Serial.println("C");
Serial.print("Humidity: ");
Serial.print(hum);
Serial.println("%");
delay(1000);
}

```

Output:



Result:

Procedure:

1. Connect the TEST pin of Flame sensor to logic state device.
2. Connect the VCC, GND of flame sensor to VCC and ground respectively.
3. Connect the OUT pin of flame sensor to pin 2 of Arduino board.
4. Connect the LED to pin 7 of Arduino Board.
5. Connect VDD and RW pin of LCD(16x4) to power supply and Ground.
6. Connect RS and EN pin of LCD to pin 13 and 12 of Arduino Board.
7. Connect D4 to D7 of LCD to pins 11 to 8 of Arduino Board.
8. Upload the provided code to the Arduino board.
9. Upload the corresponding library file to flame sensor.
10. Whenever the logic state is "0", LCD display indicate that's "NO FIRE".
11. Whenever the logic state is "1", LCD display indicate that's "FIRE Detected"
12. The variation in FIRE and NO FIRE detection is observed with LED ON and OFF.

Code:

```
// include the library code:
#include <LiquidCrystal.h> //library for LCD

// initialize the library with the numbers of the interface pins
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

// defines pins numbers
const int SensorPin = 2;
const int LEDPin = 7;

void setup()
{
  pinMode(SensorPin, INPUT); // Sets the Pin as an Input
  pinMode(LEDPin, OUTPUT); // Sets the Pin as an OUTPUT

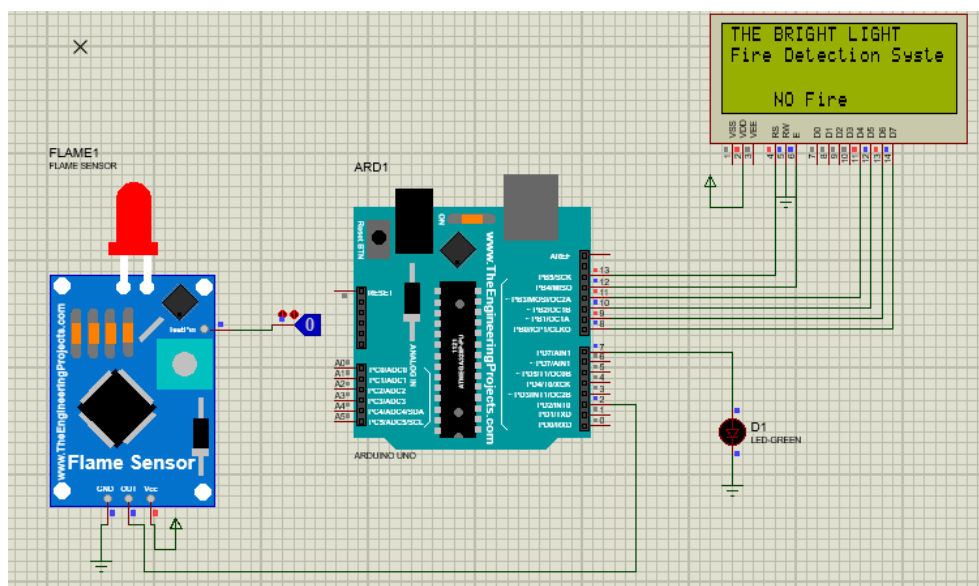
  lcd.begin(20, 4); // set up the LCD's number of columns and rows:
  lcd.setCursor(0,0); // set the cursor position:
  lcd.print(" THE BRIGHT LIGHT ");
  lcd.setCursor(0,1);
  lcd.print(" FIRE DETECTION SYS");
}
```

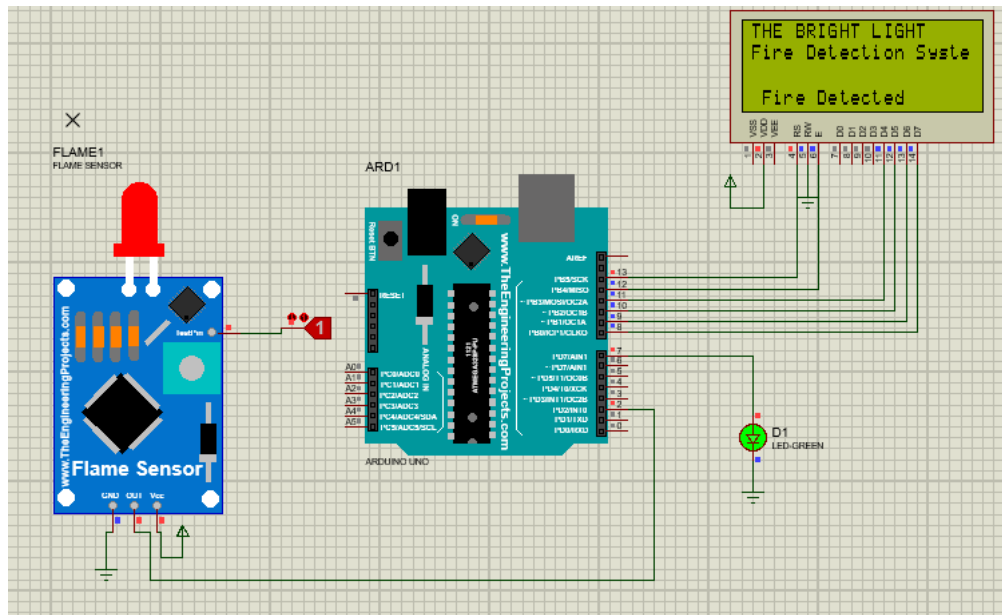
```

void loop()
{
  if(digitalRead(SensorPin) == HIGH)
  {
    digitalWrite(LEDPin,HIGH);
    // Prints Message on the LCD
    lcd.setCursor(0,3);
    lcd.print("  Fire Detected  ");
    delay(70);
    lcd.setCursor(0,3);
    lcd.print("  Fire Detected.  ");
    delay(70);
    lcd.setCursor(0,3);
    lcd.print("  Fire Detected..  ");
    delay(70);
    lcd.setCursor(0,3);
    lcd.print("  Fire Detected...  ");
    delay(70);
  }
  else
  {
    digitalWrite(LEDPin,LOW);
    // Prints Message on the LCD
    lcd.setCursor(0,3);
    lcd.print("    No Fire    ");
  }
}

```

Output:





Result:

EXP NO : 8	Interfacing Motor Using Raspberry Pi/ Arduino Based on Relay
DATE :	

Aim:

To interface a DC motor with a Raspberry Pi using a relay module and control the motor operation based on relay.

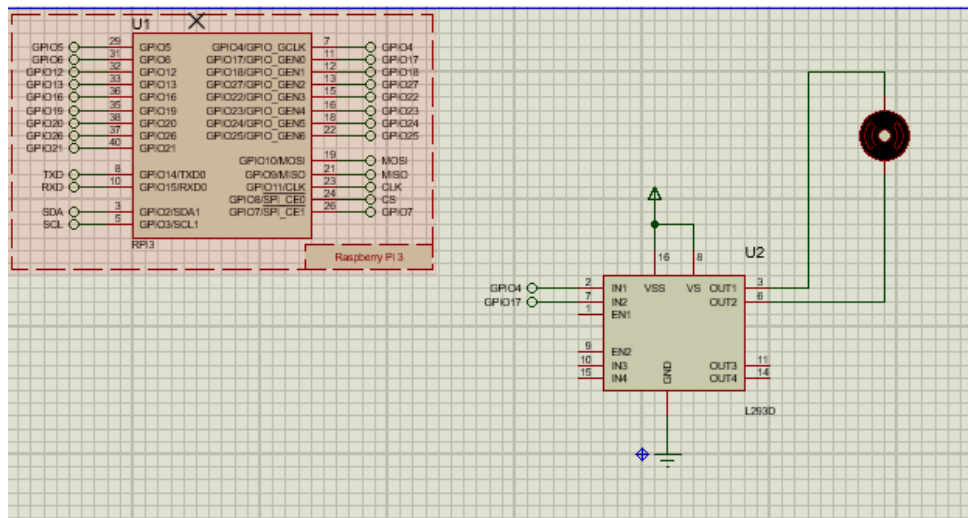
Components Required:

1. Raspberry Pi3
2. Proteus Professional 8.1 or Higher
3. DC Motor
4. Driver L239D
5. Relay
6. Lamp
7. Resistor (10k)
8. Transistor -NPN

Theory:

Interfacing a motor with a Raspberry Pi or Arduino based on temperature involves a sensor providing data to the microcontroller, which then uses a relay as an electrically-controlled switch to manage power to the motor. The core theory is that since microcontrollers cannot directly handle the high voltage and current required by most motors, a relay acts as an intermediary, isolating the low-voltage control circuit of the microcontroller from the high-power circuit of the motor. In this system, the temperature sensor (such as LM35) measures the environmental or motor temperature, and the microcontroller's program reads this data. If the temperature reading crosses a predefined threshold, the microcontroller sends a low-power signal to the relay module, causing its internal electromagnetic switch to close or open the high-power circuit, thus turning the motor on or off. This allows for automated temperature regulation, such as activating a cooling fan when a motor gets too hot.

Schematic Diagram:



Procedure:

1. While creating new project use, the option create firmware project.
2. Select the family as Raspberry Pi3 and create the schematic diagram
3. Connect the Raspberry Pi3 with Motor Driver L239D.
4. The pin GPIO 7 and 11 of Raspberry is connected with IN1 and IN2 of motor driver.
5. Connect the pin 3 and 8 of motor driver with motors
6. Connect the Vss and Vs to power supply pin and GND pin to ground.
7. Write the code the in-source code window of Proteus tool.
8. Finally run the simulation and verify the running status of motor in forward and reverse direction.

Code: 1

```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BOARD)

# Define motor pins
motor1 = 7
motor2 = 11
```

```

# Set as output
GPIO.setup(motor1, GPIO.OUT)
GPIO.setup(motor2, GPIO.OUT)

# Run motor
GPIO.output(motor1, GPIO.HIGH)
GPIO.output(motor2, GPIO.LOW)

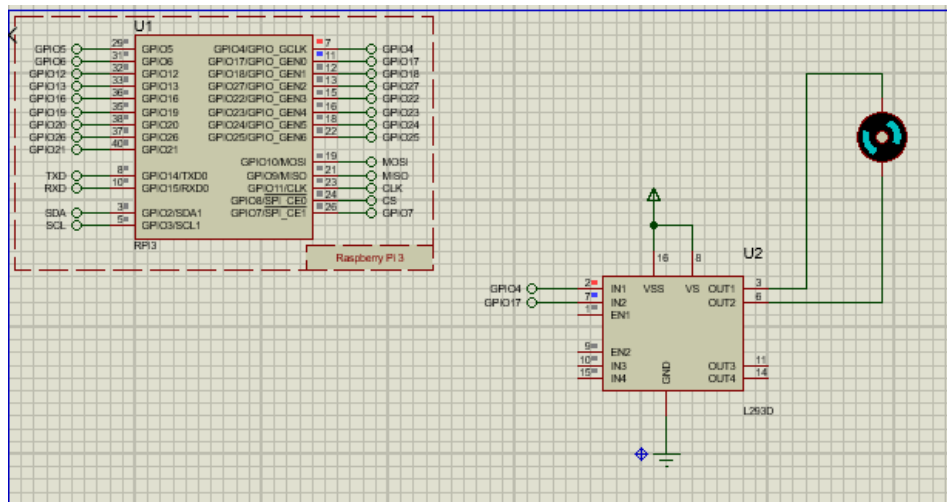
sleep(3)

# Reverse motor
GPIO.output(motor1, GPIO.LOW)
GPIO.output(motor2, GPIO.HIGH)

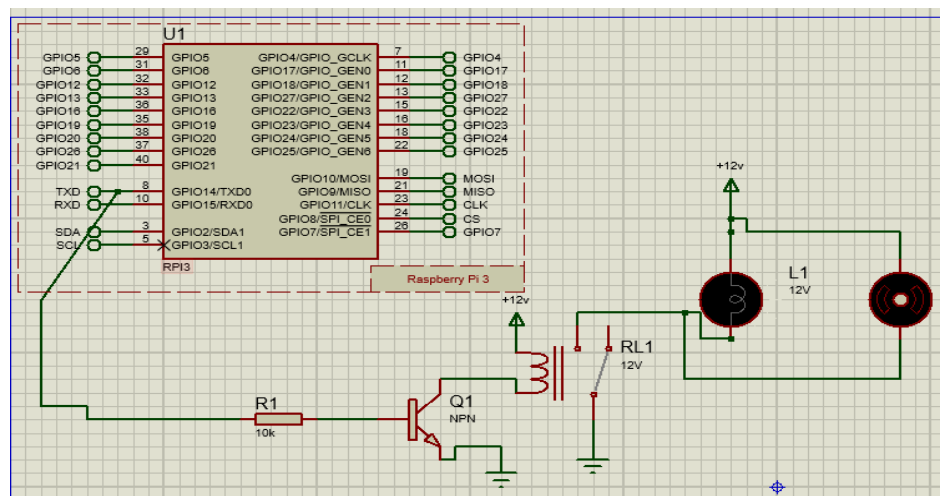
sleep(3)
GPIO.cleanup()

```

Output:



Schematic Diagram 2:



Procedure:

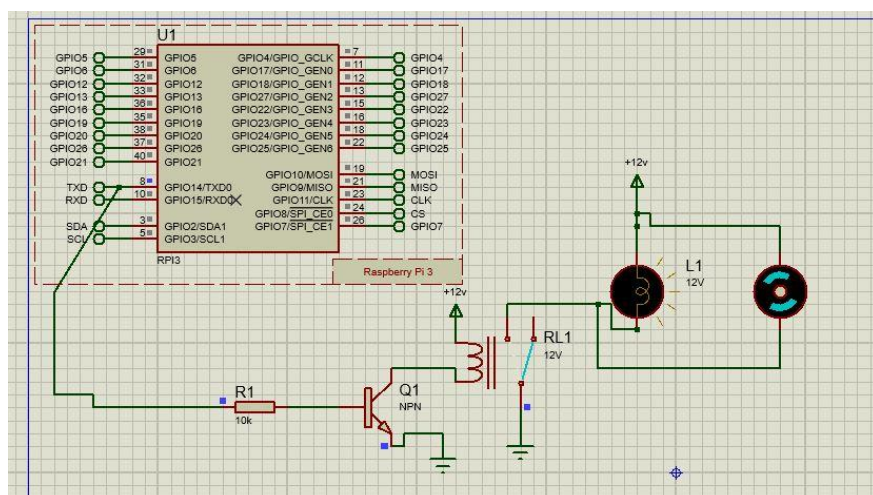
1. While creating new project use, the option create firmware project.
2. Select the family as Raspberry Pi3 and create the schematic diagram
3. Connect the Raspberry Pi3 with Resistor (10k), npn transistor and relay module.
4. Connect the relay to the lamp and with the motor
5. Write the code the in-source code window of Proteus tool.
6. Finally, run the simulation and verify the operating status of the motor and the lamp (glow), ensuring that they function correctly when the relay is switched on and off.

Code2:

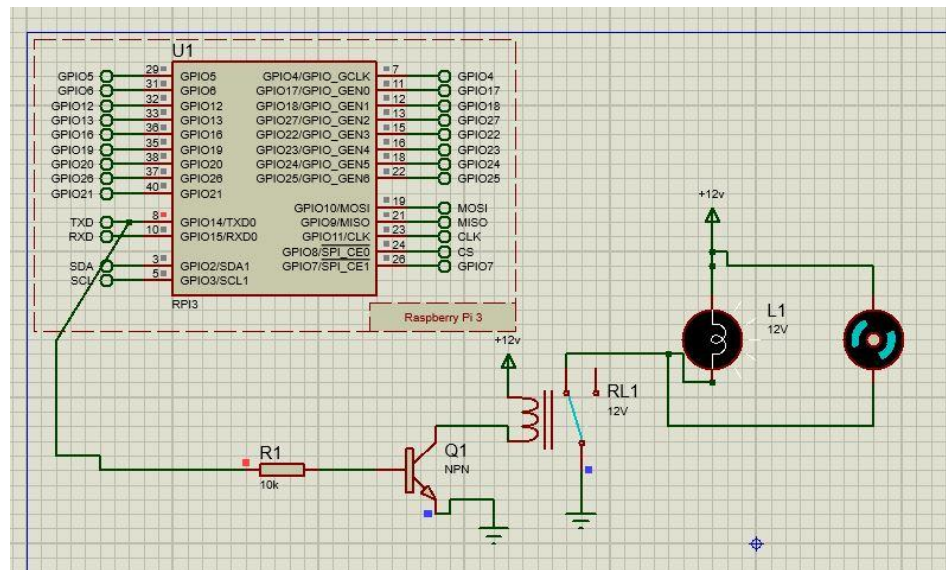
```
Import RPi.GPIO as GPIO # Import Raspberry Pi GPIO library
from time import sleep # Import the sleep function from the time module
GPIO.setwarnings(False) # Ignore warning for now
GPIO.setmode(GPIO.BOARD) # Use physical pin numbering
GPIO.setup(8, GPIO.OUT, initial=GPIO.LOW) # Set pin 8 to be an output pin and
set initial value to low (off)
while True: # Run forever
    GPIO.output(8, GPIO.HIGH) # Turn on
    sleep(1) # Sleep for 1 second
    GPIO.output(8, GPIO.LOW) # Turn off
    sleep(1) # Sleep for 1 second
```

Output:

Relay is in OFF state so the lamp is OFF state and motor turns OFF



Relay is in ON state so the lamp is ON state and motor turns ON



Result:

EXP NO : 9	Interfacing LCD with Raspberry Pi to Display Temperature and Humidity
DATE :	

Aim:

To interface a 16x2 LCD with Raspberry Pi and display real-time temperature and humidity readings from a DHT11/DHT22 sensor.

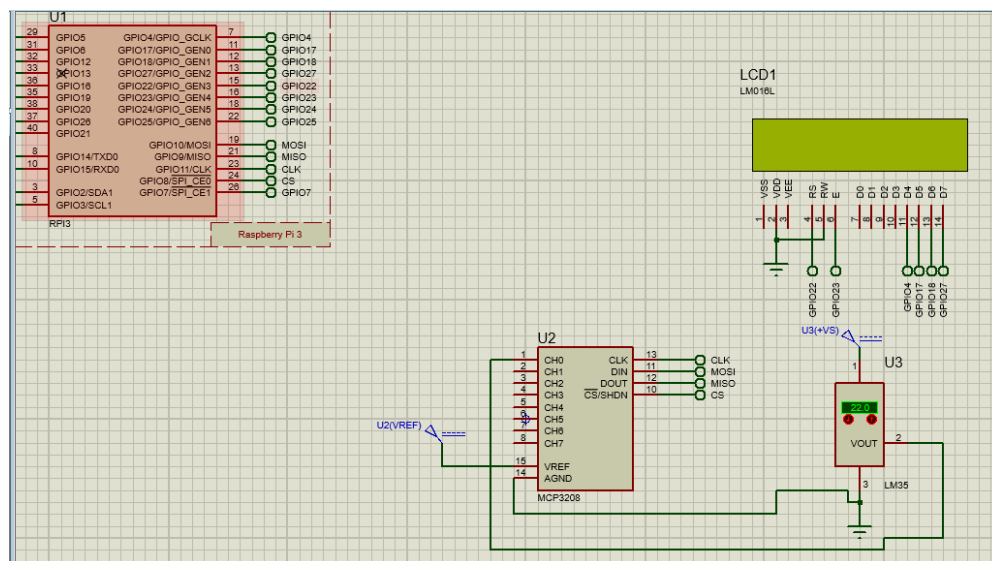
Components Required:

1. Raspberry Pi 3
2. DHT11 or DHT22 Temperature and Humidity Sensor
3. 16x2 LCD Display
4. MCP3208 driver for LM35
5. LM35 temperature Sensor
6. LCD (16x2) Display

Theory:

Interfacing an LCD with a Raspberry Pi to display temperature and humidity involves a three-part process: sensory data acquisition, microcontroller processing, and visual output. The project typically uses a DHT11 or DHT22 sensor, which captures environmental data by using a thermistor for temperature and a capacitive sensor for humidity, before transmitting the information as a digital signal over a single wire. The Raspberry Pi, running a Python script, acts as the central processor, reading this serial data from the sensor via one of its General Purpose Input/Output (GPIO) pins. To display the data, the Raspberry Pi communicates with an LCD screen, often connected via an I²C module to minimize the number of GPIO pins needed for the connection. The Python program then takes the temperature and humidity values, formats them, and sends the text to the I²C module, which in turn outputs the information to the character-based LCD screen for display.

Schematic Diagram:



Procedure:

1. Give the connections as per the schematic diagram for Raspberry Pi.
2. Connect the LM35 temperature sensor to the driver.
3. Connect the LCD (16x2) to Raspberry pi with pins given in the schematic diagram
4. Run the simulation and vary the temperature to get displayed in the LCD panel.

Python Code :

```
#!/usr/bin/python

import spidev

import time

import os

import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BOARD)

GPIO.setwarnings(False)

# Open SPI bus

spi = spidev.SpiDev()

spi.open(0,0)
```

Define GPIO to LCD mapping

LCD_RS = 15

LCD_E = 16

LCD_D4 = 7

LCD_D5 = 11

LCD_D6 = 12

LCD_D7 = 13

Define sensor channels

temp_channel = 0

'''

define pin for lcd

'''

Timing constants

E_PULSE = 0.0005

E_DELAY = 0.0005

delay = 1

GPIO.setup(LCD_E, GPIO.OUT) # E

GPIO.setup(LCD_RS, GPIO.OUT) # RS

GPIO.setup(LCD_D4, GPIO.OUT) # DB4

GPIO.setup(LCD_D5, GPIO.OUT) # DB5

GPIO.setup(LCD_D6, GPIO.OUT) # DB6

GPIO.setup(LCD_D7, GPIO.OUT) # DB7

Define some device constants

LCD_WIDTH = 16 # Maximum characters per line

LCD_CHR = True

LCD_CMD = False

LCD_LINE_1 = 0x80 # LCD RAM address for the 1st line

LCD_LINE_2 = 0xC0 # LCD RAM address for the 2nd line

'''

Function Name :lcd_init()

Function Description : this function is used to initialize lcd by sending the different commands

'''

def lcd_init():

Initialise display

lcd_byte(0x33,LCD_CMD) # 110011 Initialise

lcd_byte(0x32,LCD_CMD) # 110010 Initialise

lcd_byte(0x06,LCD_CMD) # 000110 Cursor move direction

lcd_byte(0x0C,LCD_CMD) # 001100 Display On,Cursor Off, Blink Off

lcd_byte(0x28,LCD_CMD) # 101000 Data length, number of lines, font size

lcd_byte(0x01,LCD_CMD) # 000001 Clear display

time.sleep(E_DELAY)

'''

Function Name :lcd_byte(bits ,mode)

Function Name :the main purpose of this function is to convert the byte data into bit and send to lcd port

'''

def lcd_byte(bits, mode):

Send byte to data pins

bits = data

mode = True for character

False for command

GPIO.output(LCD_RS, mode) # RS

High bits

```
GPIO.output(LCD_D4, False)
GPIO.output(LCD_D5, False)
GPIO.output(LCD_D6, False)
GPIO.output(LCD_D7, False)
if bits&0x10==0x10:
    GPIO.output(LCD_D4, True)
if bits&0x20==0x20:
    GPIO.output(LCD_D5, True)
if bits&0x40==0x40:
    GPIO.output(LCD_D6, True)
if bits&0x80==0x80:
    GPIO.output(LCD_D7, True)
```

Toggle 'Enable' pin

```
lcd_toggle_enable()
```

Low bits

```
GPIO.output(LCD_D4, False)
GPIO.output(LCD_D5, False)
GPIO.output(LCD_D6, False)
GPIO.output(LCD_D7, False)
if bits&0x01==0x01:
    GPIO.output(LCD_D4, True)
if bits&0x02==0x02:
    GPIO.output(LCD_D5, True)
```

```
if bits&0x04==0x04:
```

```
    GPIO.output(LCD_D6, True)
```

```
if bits&0x08==0x08:
```

```
    GPIO.output(LCD_D7, True)
```

```
# Toggle 'Enable' pin
```

```
lcd_toggle_enable()
```

```
'''
```

```
Function Name : lcd_toggle_enable()
```

```
Function Description: basically this is used to toggle Enable pin
```

```
'''
```

```
def lcd_toggle_enable():
```

```
# Toggle enable
```

```
time.sleep(E_DELAY)
```

```
GPIO.output(LCD_E, True)
```

```
time.sleep(E_PULSE)
```

```
GPIO.output(LCD_E, False)
```

```
time.sleep(E_DELAY)
```

```
'''
```

```
Function Name : lcd_string(message,line)
```

```
Function Description : print the data on lcd
```

```
'''
```

```
def lcd_string(message,line):
```

```
# Send string to display
```

```
message = message.ljust(LCD_WIDTH," ")
```

```
lcd_byte(line, LCD_CMD)
```

```

for i in range(LCD_WIDTH):

    lcd_byte(ord(message[i]),LCD_CHR)

# Function to read SPI data from MCP3008 chip

# Channel must be an integer 0-7

def ReadChannel(channel):

    adc = spi.xfer2([1,(8+channel)<<4,0])

    data = ((adc[1]&3) << 8) + adc[2]

    return data

# Function to calculate temperature from

# TMP36 data, rounded to specified

# number of decimal places.

def ConvertTemp(data,places):

    # ADC Value

    # (approx) Temp  Volts

    #  0   -50  0.00

    # 78   -25  0.25

    # 155    0  0.50

    # 233   25  0.75

    # 310   50  1.00

    # 465  100  1.50

    # 775  200  2.50

    # 1023 280  3.30

    temp = ((data * 330)/float(1023))

    temp = round(temp,places)

    return temp

# Define delay between readings

delay = 5

```


Output



EXP NO : 10	Case study: Industrial Plant Monitoring/Control using Arduino
DATE :	

Aim:

To study the operation and control the industrial plant operation using Arduino and GSM module.

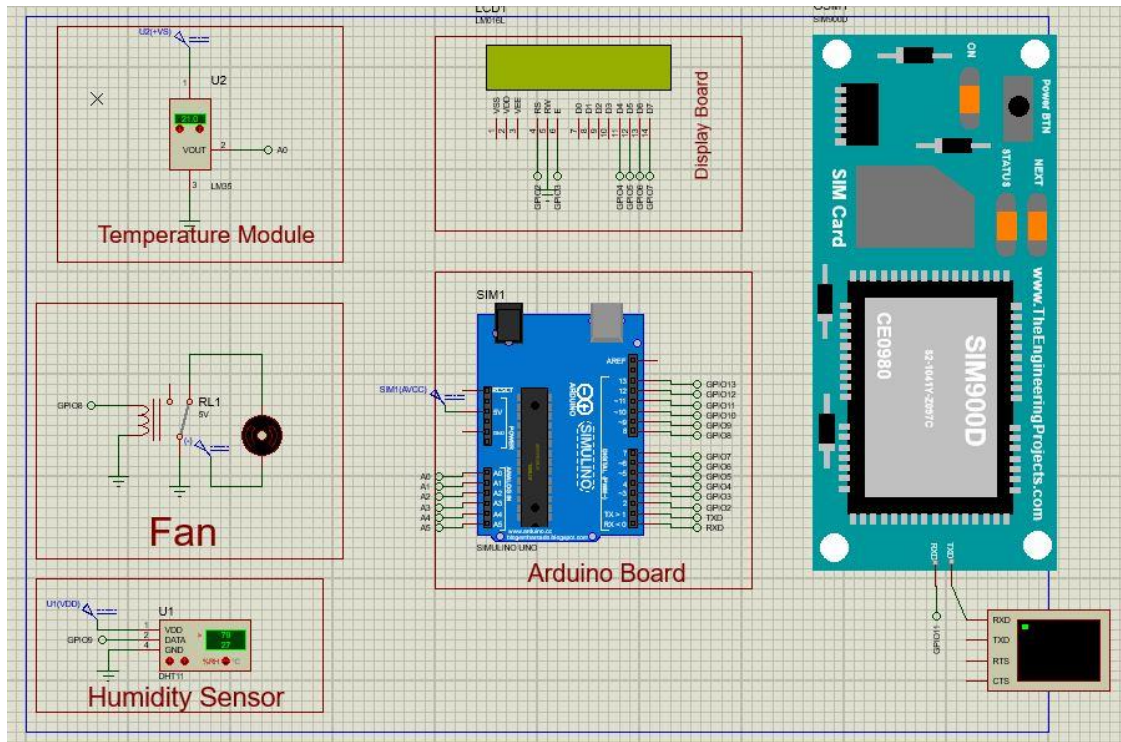
Components Required:

1. Arduino IDE 2.3.2 or Higher
2. Proteus 8.1 or Higher
3. LM35 temperature sensor
4. LCD Display (16x2)
5. GSM- SIM900 Module
6. Relay
7. Motor
8. Humidity sensor (DHT11)
9. Serial Monitor

Theory:

An industrial plant monitoring & control system using a Arduino Uno and a SIM900D GSM Module enables real-time tracking of environmental parameters like temperature and humidity. Sensors such as LM35 and DHT11 continuously measure conditions within the plant. The collected data is processed by the microcontroller and displayed locally on an LCD module for immediate observation. When predefined threshold values are exceeded, control actions like activating a fan through a relay are automatically triggered. Simultaneously, alert messages are sent to remote users via GSM communication for quick response. This system improves safety, efficiency, and remote supervision in industrial environments.

Schematic Diagram:



Procedure:

1. Connect the schematic as per the connection given
2. Vary the temperature level with temperature module LM35
3. When the temperature is below the threshold (30c), the communication alert is not initiated by the GSM module and Arduino processor
4. When the temperature is above the threshold (30c), the communication alert is initiated by the GSM module and Arduino processor
5. The variation of temperature is displayed in the LCD panel.
6. When the temperature is >30c motor/fan is turned ON else turned OFF.

Code:

```
#include "DHT.h"
#include <LiquidCrystal.h>
#include <SoftwareSerial.h>
LiquidCrystal lcd(2, 3, 4, 5, 6, 7);
const int fan = 8;
```

```

const int DHT11_Sesnor = 9;
SoftwareSerial mySerial(10,11);
const int tempPin = A0;
#define DHTTYPE DHT11
DHT dht(DHT11_Sesnor, DHTTYPE);
int val;
int humudity_value;
int cel ;
int mv;
void setup()
{
  Serial.begin(9600);
  mySerial.begin(9600);
  pinMode(fan, OUTPUT);
  lcd.begin(16, 2);
  lcd.print("Industrial Plant ");
  lcd.setCursor(0,2);
  lcd.print("Monitoring SYSTEM");
  digitalWrite(fan, HIGH);
  dht.begin();
}
void loop()
{
  readtemperature();
  readDTH11_Sesnor();
  lcd.clear();
  lcd.print("Temperature:" + String(cel) );
  lcd.setCursor(0,2);
  lcd.print("Humidity:" + String(humudity_value) );
  delay(500);
}

void readtemperature()
{
  val = analogRead(tempPin);
  mv = ( val/1024.0)*5000;
  cel = mv/10;
  Serial.print("TEMPRATURE = ");
  Serial.print(cel);
  Serial.print("*C");
  Serial.println();
  delay(500);
}

```

```

if(cel >30 )
{
    lcd.clear();
    lcd.print("Fan is On ");
    lcd.setCursor(0,2);
    lcd.print("Message Send" );
    delay(1000);
    SendMessage();
    delay(500);
    digitalWrite(fan, LOW);
}
else
{
    digitalWrite(fan, HIGH);
}
}

void readDTH11_Sesnor()
{

    // Reading temperature or humidity takes about 250 milliseconds!
    humudity_value = dht.readHumidity();

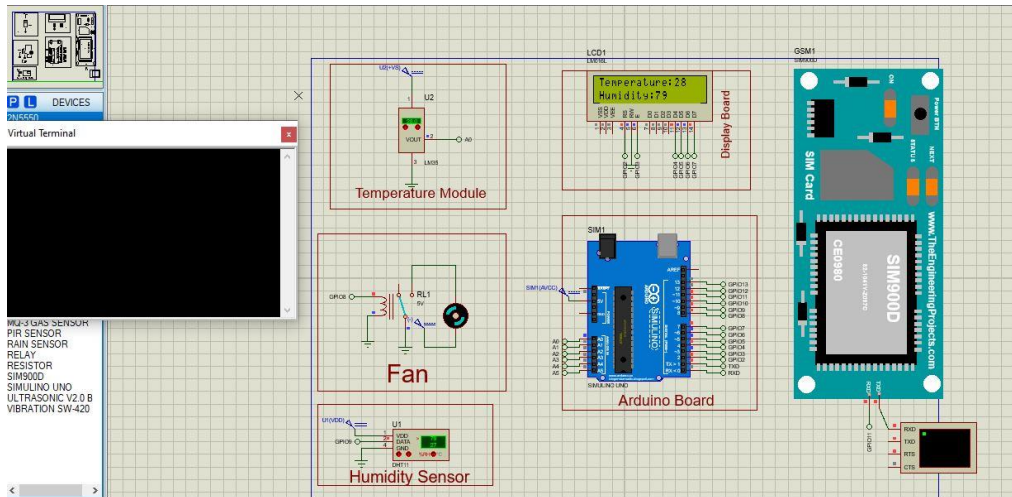
    // Check if any reads failed and exit early (to try again).
    if (isnan(humudity_value))
    {
        Serial.println(("Failed to read from DHT sensor!"));
        return;
    }
    Serial.print((" Humidity: "));
    Serial.print(humudity_value);
    Serial.print((""));

    Serial.print("\n");
    delay(1000);
}

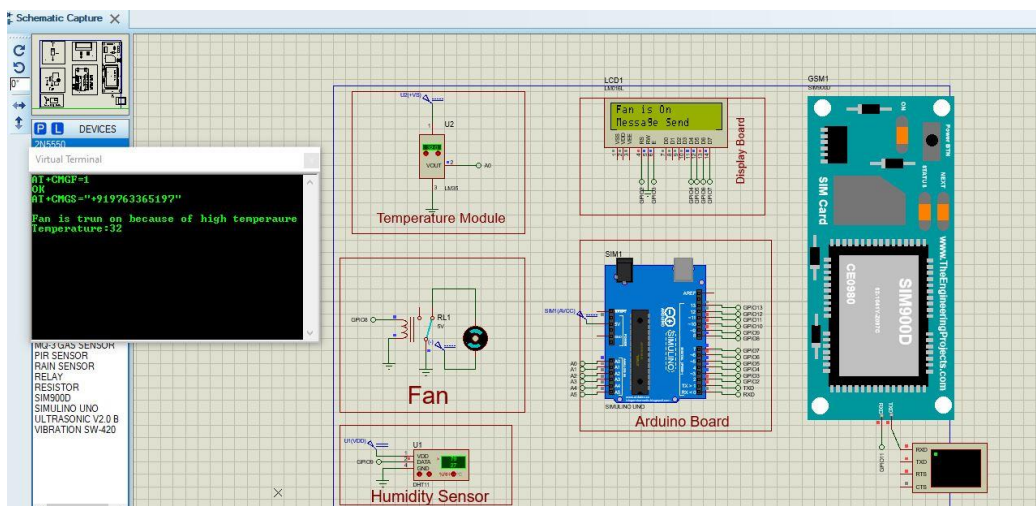
void SendMessage()
{
    mySerial.println("AT+CMGF=1"); //Sets the GSM Module in Text Mode
    delay(500); // Delay of 1000 milli seconds or 1 second
    mySerial.println("AT+CMGS="+919763365197+"\r"); // Replace x with mobile number
    delay(500);
    mySerial.println("Fan is trun on because of high temperaure in room");// The SMS text you
    want to send
    mySerial.println("Temperature:"+ String(cel));
    delay(500);
    mySerial.println((char)26);// ASCII code of CTRL+Z
    delay(500);
}

```

Output: Its shows that temperature is < 30 degrees so no commination is initiated



Its shows that temperature is < 30 degrees so no commination-initiated that can be displayed in serial monitor



Result: